F1R3FLY.IO

DESIGN SPECIFICATION

f1r3lang cost accounting

Desiderata, principles and decisions

November 6, 2026 • Version 1.0

Supersedes the stall-loop design note (revision 28) as the reference for the design.

f1r3|@ng

Contents

1 Overview **3**

2 Desiderata **3**

3 Principles **4**

4 The calculus of cost (normative) **5**

5 Price, acceptance and settlement **8**

6 Stored code and firings **9**

7 Blocks, finality and history **10**

8 Supply, tokens and hooks **12**

9 The first release **12**

A Decisions in force **14**

B Superseded decisions **16**

> **How to read this document.** Section 1 says what cost accounting is for. Section 2 states what the design must achieve, and Section 3 the principles that achieve it. Sections 4 to 7 give the design itself; Section 4 is normative, and everything else is read against it. Section 9 sets out what the first release contains and what it does not. The appendix lists the decisions in force, grouped by topic, with the numbers used in the project's records, written here in brackets, as [*n*], to keep them apart from the desiderata D1 to D10.

1 Overview

The problem. Validators of a F1R3FLY shard stall under load. Two features of the earlier design feed that stall. Every payment passed through one shared vault, so every deployment that paid for itself wrote the same state, and all of them fell into a single chain of work. And every deployment ran to completion before its effects could be combined with others, so a block's deployments had to be merged after the fact, with merge logic that grows with the load.

The remedy. Cost accounting attaches to every piece of f1r3lang code the name of who pays for it. A wallet is an independent stack of REV located on its owner's channel, not an entry in a shared map. Spending from a wallet is a communication on that wallet's channel. Two deployments that pay from different wallets therefore share nothing on their payment paths, and run concurrently. Conflicts that remain are races on shared data, which the signatures already mark.

The model. The design follows the cost-accounted rho calculus and its companion papers. A redex fires only by consuming a token from a stack held by the signature that seals it. A deployment is accepted only if a linear proof shows that its signers' stacks cover what it may spend. Payment is the communication: there is no separate settlement that turns usage into money after the fact.

> **The design in one paragraph.** Every f1r3lang expression has a rooted seal tree whose root is its deployer's wallet. A COMM consumes one phlo from the wallet at each of its surfaces' seals, and the continuation it releases runs under its own seal. All other work is priced by the cost schedule and paid from the wallet at the seal it runs under. Phlo is not a currency: a deployment's `phloPrice` says how much REV one phlo moves. REV is created only at genesis and is never destroyed; every phlo spent moves REV from a wallet to the party whose work it paid for. Blocks are ordered before they are executed, and a block is final only when a Byzantine-fault-tolerant majority has attested its post-state.

Scope of this document. It states the design as it stands, cleanly. It does not record how the design was reached; that history, the measurements behind it and the project plan remain in the stall-loop design note, which this document supersedes as the reference.

2 Desiderata

The design must meet the following requirements. Each is stated as an outcome; Section 3 shows which principles deliver it.

D1 No hot spots. Nothing on a deployment's payment path is shared with deployments that pay from other wallets. The longest chain of dependent work in a block must not grow with the number of deployments.

D2 Concurrency without merge logic. Deployments run concurrently from the block's pre-state. Conflicts are marked by the signatures and data channels they touch, and are decided once, not reconciled after the fact.

D3 Conservation. REV is created only at genesis and never destroyed. Every unit spent moves to a named counterparty.

D4 No free work. All f1r3lang work is paid for by someone. No step runs unpaid, and no shortfall is passed silently

to another party.

D5 No ambient authority. The right to draw on a wallet is named in the term. It never comes from who is running the code, from where the code sits, or from a system name any term can reach.

D6 Determinism by replay. One validator executes a block and records what happened; every other validator replays the record and reaches the same state, or rejects it.

D7 Safe finality. No block is final until its pre-state and post-state are known and attested.

D8 Bounded history. No node ever needs state or history from behind the finalization fringe, and no node ever syncs from genesis.

D9 Legacy meaning. f1r3lang written without cost annotations keeps a natural meaning: everything it does is charged to its deployer.

D10 Extension without forks. Policy (rewards, pricing, new token kinds) changes by replacing contracts behind hooks, under an authority named at genesis, not by changing consensus.

3 Principles

P1 Wallets are located REV stacks. REV lives on its owners' channels, disaggregated from any vault. A wallet's channel is computed from its owner's signature, so finding it needs no lookup. A wallet is a wrapper around a purse; they are one idea, not two [1, 79].

P2 REV is conserved. REV is created once, at genesis, from the shard's `wallets.txt` and `bonds.txt`, and no later operation creates or destroys it [2, 32].

P3 Phlo is a rate. Work costs phlo, and a deployment's `phloPrice` says how much REV one phlo moves. Phlo is never minted or held in escrow [3, 14].

P4 Two charges, two counterparties. A COMM costs one phlo, which moves to the block's proposer. All other work is priced by the cost schedule, and moves to the shard's compute wallet [4, 15, 16].

P5 No shared state on a payment path. Everything a deployment reads or writes to pay is named by its own signatures. The proposer's channel and the compute wallet receive deposits, which commute and are combined once per block.

P6 Concurrency from independence. Deployments whose footprints are independent run concurrently. Conflicts are races on shared data channels, decided by the runner and recorded.

P7 Extension by hook. Rewards, staking shares and pricing enter through hooks that default to no-ops, replaced by contracts under a multi-signature authority named at genesis [7, 31].

P8 Order, then execute. Blocks are ordered as they are proposed and executed on the ordered prefix; proposals never wait on execution.

P9 One mechanism, parametric in the token. Located stacks, gating, the journal of moves, settlement and conservation are written once for every token kind. REV is the kind whose name genesis holds, and differs from other kinds by policy, never by code path [46].

P10 Authority is named in the term. A stack may be drawn on, minted into or redirected only by a term holding a matching surface: a seal, a channel name or an unforgeable name. Identity checks in the node exist only to verify seals that appear in the term [47].

P11 Finality waits on known state. No block is final until its post-state has been attested by more than two thirds of the bonded stake [98, 99].

P12 Nothing behind the fringe. No operation of a node requires state or history behind the finalization fringe. Nodes join from an attested finalized state [116].

desideratum	delivered by
D1 No hot spots	P1, P5: wallets are separate stacks; the only shared targets take commuting deposits
D2 Concurrency	P6, P8, and the gate's per-purse proofs (§5)
D3 Conservation	P2, P3, P4: phlo only moves REV, and every move has a named counterparty
D4 No free work	The charging rule and the rooted seal tree (§4); declared bounds on stored code (§6)
D5 No ambient authority	P10 and the elaborated forms (§4)
D6 Determinism	The record and replay (§7)
D7 Safe finality	P11
D8 Bounded history	P12
D9 Legacy meaning	The rooted seal tree (§4.2)
D10 Extension	P7, P9

4 The calculus of cost (normative)

This section defines cost accounting. The annotation syntax, the rest of this document and the implementation are read against it. Any form not written here is syntactic sugar that elaborates to it [74].

4.1 Seals, membranes and wallets

A *seal* names who pays for a term: {% *P* %} [*s*] is the process *P* under the seal *s*. The seal draws on the wallet at *s*; that wallet is the purse every charge under *s* is taken from. A *membrane* is one seal of one origin: two surfaces lie under one membrane only if they have the same owner and the same origin [80].

A seal may name a public key, a compound signature $s_1 \otimes s_2$, or the hash of a process, # P, so that a contract pays for its own work. A seal may also carry its own per-firing limit, written {% *P* %} [*s* ; *n*] (§6).

4.2 The rooted seal tree

> **Every expression has a rooted seal tree.** Its outermost seal is the deployer's wallet. If a deployment carries no annotation beyond the deployer, the deployer's seal covers every subsequent expression and every reduction. Annotations only refine that tree [135].

For example, the undecorated deployment

$$\begin{aligned}\texttt{for}(p_1 \leftarrow c_1)\{\ \texttt{for}(p_{11} \leftarrow c_{11})P_{11} \mid \cdots \mid \texttt{for}(p_{1n} \leftarrow c_{1n})P_{1n}\ \} \mid \cdots \\ \mid \texttt{for}(p_m \leftarrow c_m)\{\ \texttt{for}(p_{m1} \leftarrow c_{m1})P_{m1} \mid \cdots \mid \texttt{for}(p_{mn} \leftarrow c_{mn})P_{mn}\ \} \\ \mid c_1!(Q_1) \mid \cdots \mid c_m!(Q_m) \mid c_{11}!(Q_{11}) \mid \cdots \mid c_{mn}!(Q_{mn})\end{aligned}$$

is shorthand for the same term enclosed as $\{\%\ \cdots\ \%\}[\mathit{deployerWallet}]$, which is in turn shorthand for decorating every continuation with *deployerWallet*. Every COMM of the term is then a one-party COMM under the deployer's seal, and every continuation runs under that seal.

- **Legacy code has a natural reading.** f1r3lang written before cost accounting means what it always meant: everything is charged to the deployer.
- **Decorated code refines it.** A decoration $\{\%\,P\,\%\}[t]$ replaces the seal for the subtree P, and only for that subtree.
- **Nothing runs outside every seal.** The normalizer's elaboration pass writes the root explicitly [121]. A term of a deployment that reaches the reducer with no enclosing seal violates this invariant, and its run fails. Bootstrap code is the only code without a deployer's root (§4.5).

4.3 The elaborated forms

Two parties. A COMM between a receive under s_1 and a send under s_2 is

$$\{\%\,\texttt{for}(\mathit{ptrn} \leftarrow \mathit{chan})\{\%\,P\,\%\}[t]\,\%\}[s_1] \ \mid\ \{\%\,\mathit{chan}!(Q)\,\%\}[s_2]$$

One party. A COMM whose receive and send lie under one seal s is

$$\{\%\,\texttt{for}(\mathit{ptrn} \leftarrow \mathit{chan})\{\%\,P\,\%\}[t] \mid \mathit{chan}!(Q)\,\%\}[s]$$

Defaulting. A continuation P not enclosed in its own seal uses the enclosing one: $t = s_1$ in the two-party form and $t = s$ in the one-party form.

The forms are the cost-accounted rho calculus's rule family read in f1r3lang. The one-party form is its whole-redex rule: one cell of s funds the COMM. The two-party form is its per-surface rule: each surface supplies its own cell. In both, the continuation keeps its seal t through the step, and runs, and is paid for, under t.

4.4 Firing is reduction

A *firing* is one reduction step [124]:

$$\{\%\,\texttt{for}(y \leftarrow x)\{\%\,P\,\%\}[t] \mid x!(Q)\,\%\}[s] \xrightarrow{\text{one phlo from } s} \{\%\,P\{@Q/y\}\,\%\}[t]$$

$$\{\%\,\texttt{for}(y \leftarrow x)\{\%\,P\,\%\}[t]\,\%\}[s_1] \mid \{\%\,x!(Q)\,\%\}[s_2] \xrightarrow{\text{one from } s_1,\ \text{one from } s_2} \{\%\,P\{@Q/y\}\,\%\}[t]$$

The work of that instance of P, up to its next reduction, is paid from t and bounded by t's declared limit (§6). Each later reduction it takes part in is a firing of its own, and pays its cells from its own surfaces' seals.

4.5 The charging rule

> **All f1r3lang code is charged. No code that is not f1r3lang is charged. f1r3lang code that is not decorated is charged to its deployer** [94].

- **The installer pays for its continuations.** A receive stored by a deployment keeps that deployment's seal. Its continuation runs under that seal and is charged there, not to the deployment whose send fired it [75]. A contract is a persistent receive, and follows the same rule.
- **Each surface pays its own cell.** A COMM between surfaces under different membranes costs each one phlo, at its own price; surfaces under one membrane pay once [76, 80].
- **Bootstrap is a sunk cost.** The f1r3lang that bootstraps the shard (genesis and its system deployments) is a sunk cost for the validators and is not charged.
- **Genesis code is charged to its caller.** Code installed at genesis has no deployer. When a later deployment fires it, its surface and work are charged to that deployment's deployer [81, 85].
- **Consensus code is not charged.** The validator's own code is written in Rust, not f1r3lang, and validators are paid through the COMM charges of the blocks they propose.

4.6 Two charges

Work done under a seal is paid from the wallet at that seal, at the deployment's `phloPrice`, in two ways.

charge	what it covers	phlo	REV goes to
COMM	A redex that fires: a send meets a receive, or a join fires on all its channels	One per surface	The block's proposer
Compute	Everything else: evaluation, arithmetic, method calls, substitution, and the storage of whatever a run leaves in the tuplespace	The cost schedule	The shard's compute wallet

- **Storage is charged once, when it is left.** A send and receive that meet within one run cost one phlo and no storage. Whatever a run leaves in the tuplespace pays its storage as compute. A later run that consumes it pays its COMM; nothing is refunded across runs [17, 19].
- **Not yet charged.** File I/O and the spatial matcher stay uncharged in this version, and storage pays no rent over time [18].
- **The compute wallet.** Genesis creates one compute wallet per shard, with zero balance, held by the hook authority. It receives one deposit per block [16, 31].

4.7 Compound and contract seals

- **A compound seal does not split the charge.** Under $\{\%P\%\}[s_1 \otimes s_2]$, the wallets at s_1 and at s_2 each pay the full charge for P's work. Joint authority is something every party buys: it is the cost of coordination [24]. Coordinating parties may lower what they pay by deploying at a lower `phloPrice`; the shard's price floor is not scaled, so coordination has a minimum cost [26].
- **A joint wallet pays once.** A wallet located at the compound signature itself pays once if it holds REV at the start of the block; otherwise each component pays [25, 43].

- **A contract can pay for itself.** `{%P%}[# P]` draws on the wallet at the hash of *P*, which only *P*'s own work debits and anyone may fund. Deposits are recorded per funder. A harmful contract is contained first by revoking the capabilities it needs to interact, then by refusing a funder's share, and as a last resort by a published behavioural formula behind the REV hook [27, 50].
- **Cosigners.** Every component of a compound seal signs the same envelope bytes, and the deployer's message names the required cosigners [41, 53, 63].

4.8 Membranes, data and authority

- **The membrane decides.** Data received at run time and substituted into a continuation runs under the continuation's membrane. There is no separate refusal of data received at run time [77].
- **The running membrane checks a seal in data.** A seal written inside data is checked against the membrane of the continuation that runs it, not against the deployment that wrote it. If u_2 sends `{%R%}[a]` to u_1's contract and *a* is u_1, then *R* runs and u_1 pays [130].
- **The owner check is the guard.** At run time the node refuses any seal naming a wallet that is neither a signer of the running deployment nor held by it. A seal cannot be forged or lifted into another deployment [52, 112, 130].
- **Code without a seal record has no authority.** Genesis code does not draw on its caller's wallet; the proof-of-stake contract acts through unforgeable names and handles it alone holds [54, 70, 73].
- **The seal is small.** On the term, a seal names its owner by an index into the envelope's verified signers, or by a 32-byte digest where no index applies, not by the full key [131].

5 Price, acceptance and settlement

5.1 Price

A deployment states `phloLimit`, the most phlo it may spend, and `phloPrice`, the REV it pays per phlo. One price covers both charges.

- **The shard floor is a validity rule.** A block that includes a deployment priced below the shard's minimum price is invalid. The floor is in the genesis policy [5].
- **The validator's price is its own.** Above the floor, each validator decides the price at which it includes deployments. The first version reads a fixed price from the node's configuration, through a price hook that can later be made programmable without changing consensus [5].

5.2 Acceptance by linear proof

A deployment is accepted only if a linear proof of its resource use exists: for each wallet it draws on, the REV at that wallet in the block's pre-state, less what earlier deployments in the block have committed against it, covers its share of the demand $\Delta \times$ `phloPrice`. One proof is made per purse, independently. Two deployments competing for one wallet are two proofs competing for the same hypotheses; at most one succeeds, without locks or rollback.

- **The commitment is bookkeeping.** Accepting a deployment records a commitment against each of its wallets; nothing moves. A later deployment in the block sees the earlier commitments, so the commitments on a wallet

never exceed its balance.

- **The gate commits a bound.** The commitment Δ is the smaller of `phloLimit` and a static estimate of the term's cost: the COMM-bound estimator for its COMMs, and the cost schedule's maxima over the term for the rest [122]. The run's limit equals the commitment. The estimate is conservative: it may be loose, but it is never too low [128].

5.3 Running and settling

Execution spends $\kappa \leq \Delta$ phlo. Because the commitment never left the wallet, overcharge and refund need no escrow: settlement debits $\kappa \times$ `phloPrice` and releases the rest of the commitment.

```
gate(e):    in block order, at the block pre-state R
            Delta = min(phloLimit, static estimate)
            accept iff, for each wallet w of e,
                REV(w at R) - committed(w) >= share_w(Delta) * phloPrice
            on accept: committed(w) += share_w(Delta) * phloPrice
run(e):     concurrently with every other admitted envelope, from R,
            under run limit Delta; count COMM and compute phlo per wallet
settle(e):  in block order, after the merge
            debit each w by its phlo * phloPrice; release the rest of the commitment
            credit the proposer with the COMM phlo, the compute wallet with the rest
end:        one deposit to the proposer, one to the compute wallet;
            run the REV hook; check conservation
```

- **A failed run pays its commitment.** A deployment whose run fails is debited $\Delta \times$ `phloPrice`, and its record shows Δ. This is the full charge on error that the earlier node made against `phloLimit`, applied to the gate's commitment instead; a deployment built to fail pays what it reserved. Errors found before execution, such as a term that does not parse, are charged only the parsing cost [66, 136].
- **A conflicting run still pays.** A run that loses a race at the merge is rolled back and keeps its charge.
- **Moves inside a run are order-independent.** Each native move is checked alone against the wallet's balance less its commitments, and the run's net totals per wallet are checked when it ends. An overdraw fails the whole run, which moves nothing and still pays [48].
- **The settled form is canonical.** After a block settles, each paying wallet holds one combined datum, and the proposer and the compute wallet each hold one new deposit. Nodes may batch their debits differently (the box-car size is a node setting) and still reach the same state root [6, 9].
- **Conservation is checked every block.** The sum over wallets, bonds, validator channels and the compute wallet equals the genesis supply. A scheduled full audit raises an alarm if it fails [37, 57].

6 Stored code and firings

A receive left in the tuplespace, and above all a contract, fires later inside some other deployment's run. Its work is paid by its own seal's owner, and bounded by a limit the owner declared.

- **A seal carries a per-firing limit.** $\{\%\, P \,\%\}\,[s\,;\,n]$ bounds each firing of P by n phlo. A seal without a declared limit takes the signed `phloLimit` of the deployment that wrote it, never the limit of a seal around it [120, 127].
- **The innermost seal bounds a firing.** In $\{\%\,\texttt{for}(y \leftarrow x)\{\%\, P \,\%\}\,[t\,;\,n]\,\%\}\,[s_1\,;\,m]$, the fired instance of P is bounded

by n. If P has no seal of its own, it is under s_1 by the rooted seal tree, and m bounds it. A seal deeper inside P adds no separate running total; its limit applies when it is itself stored and fires [133].

- **An unfunded receive does not fire.** A stored receive is a candidate for a match only if its owner's wallet covers its cell and its firing's bound. The test reads only pre-state values, so play and replay choose alike, and replay re-checks every recorded COMM [79, 120].
- **Bounds are held, and shared fairly.** A firing's bound is held against its seal owner's wallet, and against any owner its body names directly, until the run ends. When n deployments are admitted to a block, each run may hold at most $\mathit{free}(W)/n$ of a wallet W, so runs together can never overdraw it [125, 126, 129].
- **Overcharge and refund.** The owner is debited what the firing used and the rest of its bound is released.

6.1 Errors inside a firing

> **Any nested error fails the entire enclosing run** [132]. This includes a firing that exhausts its declared bound, and any other error inside a firing, such as a refused seal.

Take

$$\{\%\,\mathtt{for}(p \leftarrow c)\{\%\,\{\%\,\mathtt{for}(y_1 \leftarrow x_1)\{\%\,P_1\,\%\}\,\%\}\,[u_{11}]$$
$$\mid \{\%\,\mathtt{for}(y_2 \leftarrow x_2)\{\%\,P_2\,\%\}\,\%\}\,[u_{12}]\,\%\}\,[t]\,\%\}\,[s_1]$$
$$\mid \{\%\,c!(Q)\,\%\}\,[s_2] \mid \{\%\,x_1!(Q_1)\,\%\}\,[u_{21}] \mid \{\%\,x_2!(Q_2)\,\%\}\,[u_{22}]$$

If the firing paid by u_{11} and u_{21} runs out of phlo, the whole run fails; the outer comprehension does not carry on with the firing of P_2. Charging does not change because of the failure: each seal's owner pays the bound its firing used, the deployment pays its commitment (§5), and the run's effects are discarded. Replay recomputes each firing's cost and rejects a record that differs.

Within a run, a firing's effects are buffered until it completes, so that charges are deterministic: without the buffer, a firing could perform some operations before exceeding its bound, and which ones would depend on evaluation order. Isolating a nested failure, so that the outer computation can continue, is future work.

7 Blocks, finality and history

7.1 Order, then execute

Blocks are ordered as they are proposed. Execution follows the certified order and runs each block in four steps:

1. **Gate** every deployment at the block's pre-state, in block order, accumulating commitments.
2. **Run** every admitted deployment concurrently from the pre-state.
3. **Merge** in block order; a conflicting run is rolled back and keeps its charge.
4. **Settle** in block order, in the canonical form; run the REV hook; check conservation.

Only the run is expensive, and it is the concurrent step.

The record. One validator executes the block and produces its record: each deployment's gate verdict and commitment, its outcome, the COMM and compute phlo per wallet, the REV moved, and what replay needs. Play may be nondeterministic where races are; the record fixes the outcome, and every other validator replays it and confirms the same state change [89].

7.2 Finality waits on known state

> **P11.** No block is final until its pre-state and post-state are known. Blocks are ordered as soon as they are proposed, executed on the ordered prefix, and counted as final only once more than two thirds of the bonded stake has attested their post-state [98].

P11 is realized by splitting finality in two [99]:

- **The order floor.** The finalizer's order result is an order floor that execution follows. A configured bound limits how far it may run ahead of attested state; past it, proposals stop growing the backlog.
- **State finality.** A block is final when it is at or below the order floor and attestations from more than two thirds of the bonded stake name the same post-state root at its position or later.
- **Void, not reject.** A block above a certified order cannot be removed without letting one faulty block remove every honest block after it. A block found invalid by its execution is therefore *void*: its post-state is its pre-state, its deployments return to the pool, and evidence names its proposer. Later blocks run on it normally. The reasons are recorded in the canonical record: invalid slash records, unreadable terms, and a close that fails. A firing set off by the close that exceeds its bound fails the close, so its block is void for that reason, and the record replays [109, 134].
- **Void blocks are identifiable.** A void block stays in the justifications by its hash, and "void" is part of its record, checked in replay and attested. A deployment is charged once, in the block that runs it; one that already ran in a non-void block is refused at execution [109].
- **Slashing a void.** Only invalidity visible from the proposer's own justifications is slashable. A void caused by state the proposer could not see is recorded, not slashed [101].

7.3 Nothing behind the fringe

> **P12.** No operation of a node may require state or history behind the finalization fringe. Past data may be rolled off to slower storage or pruned. A node joins from an attested finalized state; it never replays from genesis [116].

A chain whose nodes must sync from genesis centralizes as it grows: only well-resourced institutions can afford to verify from the beginning. Under P11 the finalized root is agreed by a quorum of stake, so syncing from it trusts no single party. A joining node starts from a recent checkpoint in its configuration, verifies forward through the validator-set handoffs at epoch boundaries, each certified by more than two thirds of the previous set, and checks the current finalized block's quorum [117]. Every rebuild and replay is likewise bounded by the fringe: an in-place upgrade replays each block it needs from that block's own pre-state, never from genesis [115].

7.4 The close

Each block ends with the proof-of-stake close, which is f1r3lang and so, by the charging rule, is charged: its undecorated code to the proposer, and continuations it fires to their own seals' owners. The close is linear in the number of

bonds and of withdrawers [107, 110]. It is priced at the shard's price floor [106].

8 Supply, tokens and hooks

- **Genesis.** Genesis creates one located stack per line of `wallets.txt`, the bonds of `bonds.txt` on native bond channels, and an empty compute wallet. This is the only creation of REV. The genesis mint is unreachable from any later term [29, 32, 40].
- **Bonds.** Bonding and release are native moves between a validator's wallet and its bond channel; slashing moves a bond to the compute wallet. Release and slashing require a name only the proof-of-stake contract holds [40, 54, 72].
- **Token kinds.** Token creation is programmable: a contract may mint stacks of its own token kinds, named by an unforgeable name it holds. REV is the kind whose name genesis holds, so no contract can mint it, and REV is the only staking token [28, 42].
- **The REV hook.** Genesis installs a hook, called after each block's settlement and at epoch boundaries, that does nothing. Rewards and staking shares are contracts that replace it. A hook can move REV but cannot create it, because conservation is checked after it runs [2].
- **Hook authority.** Replacing a hook, or spending from the compute wallet, requires a deployment signed by the proof-of-stake multi-signature keys and quorum named at genesis [7, 31, 55].
- **No migration.** F1R3node is software from which others build blockchains. There is no legacy chain to preserve and no balances to migrate [8].

9 The first release

The first release hands the team a branch that implements the design end to end [78]. Its scope is frozen; the following rulings set it apart from the full design [118].

- **The close is recorded, not charged.** Each close records its cost κ in the block record; charging it is enforced in a later release [119].
- **Joining is a later track.** P12 stands, and the branch has been audited against it, but joining from an attested state with a checkpoint [117] is a sync protocol of its own and follows the release [118].
- **Every deployment is elaborated.** The normalizer puts every deployment into the elaborated forms before it runs, so the reducer reads seals from the term [121].

Known issues

- The gate's static estimate is loose (15 to 170 times the work done), so most deployments still commit their `phloLimit` [128].
- In a full block, a contract with a small wallet may skip firings, because of the $\mathit{free}(W)/n$ share [125].

- The native balance address appears in source text.
- The setting `deploy-verified-encodings-cap` is ignored.
- One shared test fails when tests run as root.
- One segmentation fault, seen once, has not been reproduced.
- Formatting against the pinned toolchain is left to continuous integration.

Future work

- Charging the close, with the pre-charge design kept on record.
- Joining from an attested state (P12).
- Tightening the gate’s estimate from measured data.
- Isolating nested failures so the outer computation can continue.
- Rent for stored data over time, and charges for file I/O and the spatial matcher.
- A programmable price hook.

A Decisions in force

The decisions below are those that define the design, grouped by topic. Numbers are those of the project's records. Decisions about the project plan, branches and schedule are omitted, as are decisions later superseded; Appendix B lists the latter.

no.	decision
Value and supply	
1	Wallets are located REV stacks on their owners' channels, disaggregated from the vault; moving REV is native.
2	REV is conserved and minted only at genesis; further behaviour enters through a hook that defaults to a no-op.
3	Phlo is the rate at which wallet REV is spent; it is not minted and there is no escrow.
4	REV consumed by COMMs moves to the proposer the block names; no burn, no fixed fee.
14	A COMM costs one phlo; `phloPrice` is the REV spent per phlo, for both charges.
15	Non-COMM work under {% *P* %}[*s*] is priced by the cost schedule and debited from the wallet at *s*.
16	Compute charges go to one compute wallet per shard, held by the hook authority.
28	Token creation is programmable; REV is the only staking token.
29	REV is minted at genesis inside a genesis-only process that later terms cannot reach.
32	Genesis supply is `wallets.txt` plus `bonds.txt`; bonds live on native stacks.
40	Bonds live on a native bond channel per validator; slashing moves the bond to the compute wallet.
42	A stack signed with a kind's name mints; signed with a read-only bundle of it, transfers.
46	Cost accounting is one mechanism, parametric in token kind.
Price and settlement	
5	The shard minimum price is a validity floor; the validator's price is a fixed node price behind a hook.
6	Box-car batching of debits is a per-node setting.
9	The settled form is canonical, so box-car size never changes the state root.
17	What a run leaves in the tuplespace pays its storage as compute; rent is deferred.
18	File I/O and pattern matching stay uncharged in this version.
19	No cross-run storage refund; the COMM's one phlo replaces the COMM event-storage charge.
37	Conservation is checked per block in play and replay; a full audit runs on a schedule.
48	Native moves inside a run are admitted order-independently; an overdraw fails the whole run, which still pays.
57	A failed scheduled audit raises an alarm; the per-block check is the enforcement.
66	A term that fails to normalize is charged to the deployer alone.
122	The gate commits the smaller of `phloLimit` and a static estimate of the term.
128	The gate's estimate is conservative for the first release and is tightened later.
136	A failed run pays its commitment: debit and recorded cost are both Δ.
The calculus of cost	
24	A compound seal does not split the charge; each component pays in full.
25	A joint wallet keyed by the compound signature pays once.
26	Coordinating parties may lower `phloPrice`; the floor is not scaled.
27	`# P` seals let a contract pay for its own work; containment by revocation, refusal of a funder's share, then a behavioural formula.
43	A deployment's payers are the signatures it names; the deployer pays when it names none.
49	A sealed region's work is charged to its owner at the owner's price.
74	The elaborated forms are the normative definition; every other form is sugar.
75	A continuation without its own seal runs under, and is paid by, its enclosing seal.
76	Each surface of a COMM pays its own cell; surfaces under one seal pay once.

no.	decision
79	A wallet is a wrapper around a purse; an unfunded stored receive is not a match candidate.
80	Surfaces pay one cell only under one membrane (same owner and origin).
81	A surface with no seal record is charged to the client whose run fired it.
85	A COMM between a client's send and genesis code counts two cells, both charged to the client.
94	All f1r3lang is charged; non-f1r3lang is not; undecorated code is charged to its deployer.
121	An elaboration pass in the normalizer puts every deployment into the elaborated forms.
124	Firing is reduction: one phlo from s, or one each from s_1 and s_2.
135	Every expression has a rooted seal tree whose root is the deployer's wallet; annotations refine it.
Stored code	
120	Stored code is bounded by a per-firing limit in its seal; overcharge and refund.
125	Each run may hold at most $free(W)/n$ of a seal owner's wallet.
126	A firing's bound is held until its run ends.
127	Per-seal limit `{%P%}[s; n]`, defaulting to the signed `phloLimit`.
129	A firing's bound is held by its seal owner and the owners its body names.
132	Any nested error, including exhausting a bound, fails the entire enclosing run.
133	The innermost seal bounds a firing; an unsealed continuation takes the enclosing limit.
Authority	
41	Compound seals authorize through a cosigned envelope; terms name cosigners by index.
47	Authority over a stack is named in the term (P10).
50	Refusing a funder's share of a `# P` wallet needs the hook authority's seal.
52	Sealed code may run in an envelope its sealer did not sign; its origin is recorded by the reducer alone.
53	A cosignature covers the whole envelope.
54	Bond release and slashing need a name only the proof-of-stake contract holds.
55	The hook spends the compute wallet under its own seal; replacement needs the hook authority.
63	The deployer's signed message names the required cosigners.
70	Code with no seal record has no authority.
72	A validator bonds natively in its own sealed code.
73	Proof-of-stake withdrawal requires an unforgeable handle issued at registration.
77	Received data runs under the continuation's membrane; no separate refusal.
112	A seal bound only at run time is guarded by the run-time owner check.
130	A seal in data is checked against the running membrane.
131	A seal names its owner by a signer index or a 32-byte digest.
Blocks, finality and history	
7	Hook updates need a multi-signature from addresses named at genesis.
31	The hook authority is the proof-of-stake multi-signature keys and quorum.
89	Play may be nondeterministic; replay follows the record.
98	P11: no block is final until its state is known.
99	P11 is realized by an order floor and state finality; invalid blocks are void.
101	Only invalidity visible from the proposer's justifications is slashable.
105	The close starts an epoch at the first block at or past the boundary that runs one.
106	The close is priced at the shard's price floor.
107	The proof-of-stake close is linear in the number of bonds.
109	Void blocks are identifiable; a deployment is charged once, in the block that runs it.
110	The withdrawer loops of the close are rewritten to be linear.
115	Upgrades replay from each block's own pre-state, never from genesis.
116	P12: nothing behind the finalization fringe.
117	Joining starts from a configured checkpoint and verifies forward through certified handoffs.
119	In the first release the close is recorded, not charged.
134	A close-fired firing over its bound fails the close; the block is void and replays.

no.	decision
Scope	
8	No legacy chain and no migration.
39	The earlier node's legacy mode stays; the vault path is replaced on cost-accounted shards only.
78	The branch ships only when it implements the elaborated forms end to end.
118	Scope of the first release is frozen; joining follows it.

B Superseded decisions

decisions	what they said	replaced by
45	One native payer per wallet per block	49
59 (metering)	Stored code meters to its caller	75
62, 67, 68, 71	Seals and stacks in data received at run time are refused	77, 130
86, 89 (gating), 103, 104	Work under a membrane is admitted step by step, with refusals in the trace	120
92, 96, 97, 102, 108, 111, 113	The close is exempt, or charged after settlement, or pre-charged to the proposer	119 (for the first release)